

ML model to predict ROAS (Return on Ad Spend)

Save model using Joblib Simple Tkinter desktop app UI Jupyter Notebook training workflow

1. IMPORT LIBRARIES

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

import joblib
```

2. LOAD DATASET

```
In [2]: df = pd.read_excel("FREELANCE6 advanced_social_media_marketing_20000.xlsx")
df.head()
```

Out[2]:

	date	campaign_id	channel	campaign_type	roas	customer_segment	impressions	clicks	li
0	2025-06-15	CAM_9779	Instagram	Retargeting	11.79	Gen Z	102263	4125	72
1	2025-02-18	CAM_4517	TikTok	Traffic	6.73	Gen Z	16131	6810	3
2	2025-01-20	CAM_4943	Meta	Retargeting	16.98	High Value	73864	2676	15
3	2025-02-16	CAM_3028	LinkedIn	Retargeting	4.93	Gen Z	63858	6360	2
4	2025-04-25	CAM_7499	LinkedIn	Awareness	0.05	Low Value	41436	1273	8

3. FEATURE SELECTION

TARGET → ROAS

```
In [3]: target = "roas"

features = [
    "impressions",
    "clicks",
    "likes",
    "comments",
    "shares",
    "cost_of_ad",
    "channel",
    "campaign_type",
    "customer_segment",
    "traffic_source"
]
```

4. SPLIT DATA

```
In [4]: X = df[features]
        y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

5. PREPROCESSING + MODEL PIPELINE

```
In [5]: categorical_features = ["channel", "campaign_type", "customer_segment", "traffic_source"]
numeric_features = ["impressions", "clicks", "likes", "comments", "shares", "cost_of_ad"]

preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
        ("num", "passthrough", numeric_features)
    ]
)

model = RandomForestRegressor(
    n_estimators=150,
    random_state=42
)

pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", model)
])
```

6. TRAIN MODEL

```
In [6]: pipeline.fit(X_train, y_train)
```

```
Out[6]: Pipeline(steps=[('preprocessor',
                          ColumnTransformer(transformers=[('cat',
                                                            OneHotEncoder(handle_unknown
                                                            = 'ignore'),
                                                            [
                                                                'channel', 'campaign_type',
                                                                'customer_segment',
                                                                'traffic_source'
                                                            ]),
                                                            ('num', 'passthrough',
                                                            [
                                                                'impressions', 'clicks',
                                                                'likes', 'comments',
                                                                'shares', 'cost_of_ad'
                                                            ])])),
                          ('model',
                           RandomForestRegressor(n_estimators=150, random_state=42))])
```

7. EVALUATION

```
In [7]: y_pred = pipeline.predict(X_test)

print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
RMSE: 3.13406348788757
R2 Score: 0.516893230614049
```

8. SAVE MODEL (JOBLIB)

```
In [9]: joblib.dump(pipeline, "roi_predictor_model.pkl")  
        print("Model saved successfully!")
```

Model saved successfully!

TKINTER APP (ROI PREDICTOR)

FULL TKINTER CODE

```
In [11]: import tkinter as tk
from tkinter import ttk
import joblib
import numpy as np

# Load trained model
model = joblib.load("roi_predictor_model.pkl")

# ----- GUI ----- #

app = tk.Tk()
app.title("Ad Campaign ROI Predictor")
app.geometry("500x600")

# Inputs
def predict_roi():
    input_data = np.array([[
        float(impressions.get()),
        float(clicks.get()),
        float(likes.get()),
        float(comments.get()),
        float(shares.get()),
        float(cost.get()),
        channel.get(),
        campaign_type.get(),
        segment.get(),
        traffic.get()
    ]])

    prediction = model.predict(input_data)[0]

    result_label.config(text=f"Predicted ROAS: {round(prediction,2)}")

# Labels + Inputs
tk.Label(app, text="Impressions").pack()
impressions = tk.Entry(app)
impressions.pack()

tk.Label(app, text="Clicks").pack()
clicks = tk.Entry(app)
clicks.pack()

tk.Label(app, text="Likes").pack()
likes = tk.Entry(app)
likes.pack()

tk.Label(app, text="Comments").pack()
comments = tk.Entry(app)
comments.pack()

tk.Label(app, text="Shares").pack()
shares = tk.Entry(app)
shares.pack()

tk.Label(app, text="Cost of Ad").pack()
cost = tk.Entry(app)
```

```
cost.pack()

# Dropdowns
tk.Label(app, text="Channel").pack()
channel = ttk.Combobox(app, values=["Meta", "Instagram", "TikTok", "YouTube", "LinkedIn"])
channel.pack()

tk.Label(app, text="Campaign Type").pack()
campaign_type = ttk.Combobox(app, values=["Awareness", "Engagement", "Conversion", "Lead Generation"])
campaign_type.pack()

tk.Label(app, text="Customer Segment").pack()
segment = ttk.Combobox(app, values=["Gen Z", "Millennials", "Gen X", "High Value", "Low Value"])
segment.pack()

tk.Label(app, text="Traffic Source").pack()
traffic = ttk.Combobox(app, values=["Organic", "Paid Social", "Referral", "Direct", "Email"])
traffic.pack()

# Button
tk.Button(app, text="Predict ROI (ROAS)", command=predict_roi).pack(pady=10)

# Output
result_label = tk.Label(app, text="", font=("Arial", 14))
result_label.pack()

app.mainloop()
```

In []: